

ПАМЯТКА

Программирование в симуляторе Пионера

Lua, Python и Blockly на одной странице: как запустить первую программу за пять минут и не наступить на самые частые грабли.

Адрес: <https://simulator.sakoryagin.ru>

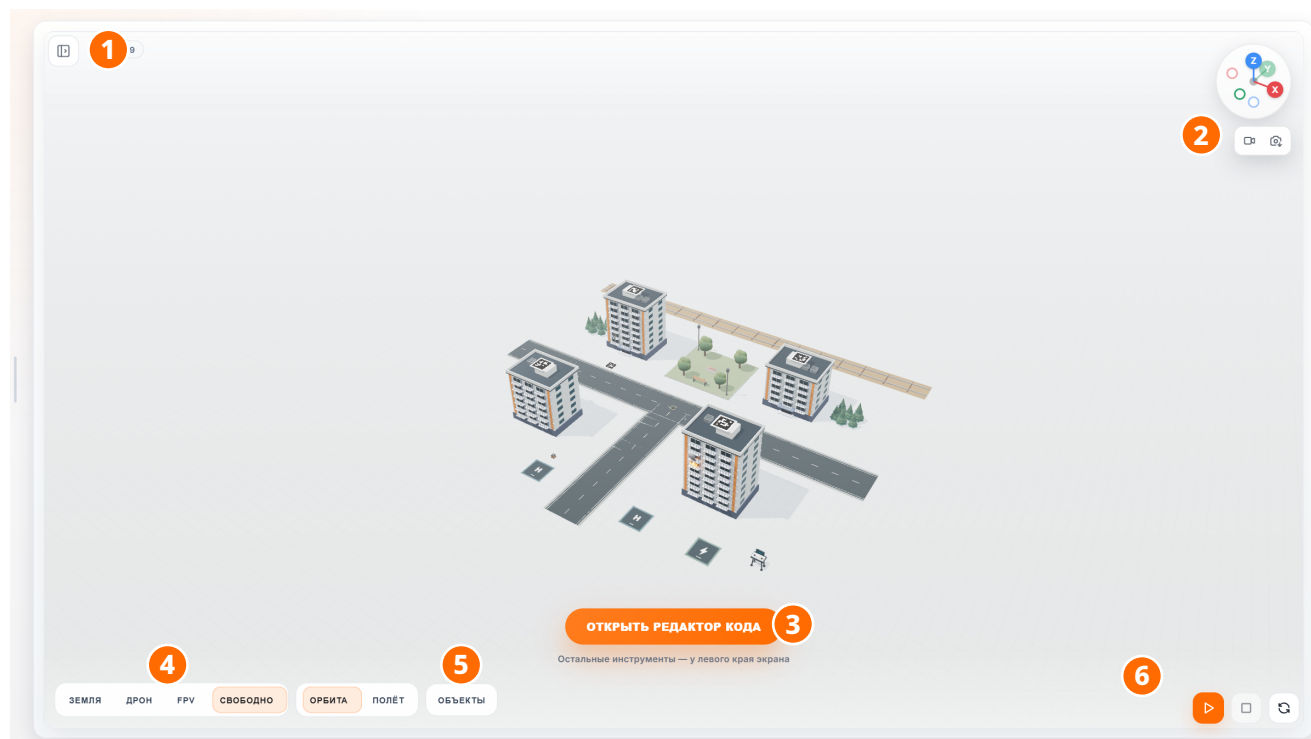
Что внутри

1. Быстрый старт
2. Первая программа на Lua
3. Координаты: где точка
4. Несколько точек подряд
5. Таймеры, светодиоды, датчики
6. Python в браузере
7. Blockly: блоки вместо кода
8. Камеры дрона
9. Python со своего компьютера
10. Частые ошибки
11. Шпаргалка: Lua и Python

Три главные мысли

- Дрон — это автопилот: вы *отдаёте команду* и *ждёте события*, что она выполнена.
- В Lua ждут через `callback(event)`, в Python — опросом `while not ...`.
- Координаты точек — метры в системе сцены, их показывает `Ctrl+ЛКМ`.

1 Быстрый старт



- | | |
|--|---|
| 1 Панели: код, телеметрия, рой, менеджер сцены, руководство, настройки. | 4 Камера: над землёй, за дроном, от лица дрона (FPV), свободная. |
| 2 Компас осей (щелчок по оси — вид с неё) и окна камер дрона. | 5 Расстановка объектов на сцене: дома, маркеры, машины. |
| 3 «Открыть редактор кода» — сюда пишется программа. | 6 «Запустить», «Стоп» и «Сбросить» (дрон возвращается на старт). |

Пять шагов до полёта: открыть сайт → «Открыть редактор кода» → выбрать язык в левом верхнем списке (Lua, Python или Blockly) → вставить пример из раздела 2 → нажать «Запустить». Пока открыт редактор, сцена скрыта: закройте его кнопкой «Код» на панели слева — программа продолжит работать, и вы увидите полёт.

Совет Готовые сцены (жилой квартал, Геоскан Арена, гоночная трасса) — в «Менеджере сцен». Вращать камеру — левой кнопкой мыши, сдвигать — правой, приближать — колёсиком.

2 Первая программа на Lua

Автопилот Пионера работает на событиях. Программа отдаёт команду и заканчивает, а когда дрон её выполнит, автопилот сам вызывает вашу функцию `callback(event)` и сообщает, что произошло. Следующую команду отдают *там*, по нужному событию.

Программа в редакторе. Её текст — ниже, его можно скопировать из PDF.

```
ap.push(Ev.MCE_PREFLIGHT) -- запустить моторы

function callback(event)
  if event == Ev.ENGINES_STARTED then
    ap.push(Ev.MCE_TAKEOFF) -- моторы запущены: взлетаем
  end
  if event == Ev.TAKEOFF_COMPLETE then
    ap.goToLocalPoint(2, 1, 1.5) -- взлетели: летим в точку (x, y, z в метрах)
  end
  if event == Ev.POINT_REACHED then
    ap.push(Ev.MCE_LANDING) -- долетели: садимся
  end
end
```



Та же программа в полёте, камера «Дрон». Голубая линия — след полёта.

Команды автопилоту — `ap.push(...)`

<code>Ev.MCE_PREFLIGHT</code>	запустить моторы
<code>Ev.MCE_TAKEOFF</code>	взлететь
<code>Ev.MCE_LANDING</code>	сесть
<code>Ev.ENGINES_DISARM</code>	выключить моторы

События от автопилота — в `callback`

<code>Ev.ENGINES_STARTED</code>	моторы запущены
<code>Ev.TAKEOFF_COMPLETE</code>	взлёт закончен
<code>Ev.POINT_REACHED</code>	точка достигнута
<code>Ev.COPTER_LANDED</code>	дрон сел
<code>Ev.LOW_VOLTAGE2</code>	батарея садится

Функция `callback` обязательна Даже если события не нужны, объявите пустую `function callback(event) end`. Без неё автопилот выполнит только первую команду миссии, а остальные проигнорирует.

3 Координаты: где точка



`Ctrl` + щелчок по земле или по объекту показывает координаты точки.

- Координаты — в **метрах**, в системе сцены. Ось **Z смотрит вверх**, X и Y лежат на земле; их направления показывает компас в правом верхнем углу.
- Эти же числа понимают `ap.goToLocalPoint(x, y, z)` и `go_to_local_point` в Python, и их же возвращают `Sensors.lpsPosition()` и `get_local_position_lps()`: щёлкнули с `Ctrl` — подставили в программу.
- Показанное **Z — высота поверхности**, по которой щёлкнули. Лететь надо выше: над крышей дома на высоте 13 м задавайте, например, $z = 14$.
- Курс (`ap.updateYaw, yaw` в Python) — в **радианах**: $\pi/2 \approx 1.57$ — четверть оборота.

4 Несколько точек подряд

Каждая точка заканчивается одним и тем же событием POINT_REACHED. Чтобы понять, *какая* точка достигнута, заведите счётчик шагов — так сделаны официальные примеры Geoscan.

```
local points = {{0, 0, 1}, {2, 0, 1}, {2, 2, 1.5}, {0, 2, 1}}
local step = 0

ap.push(Ev.MCE_PREFLIGHT)

local function nextPoint()
    step = step + 1
    if step <= #points then
        ap.goToLocalPoint(points[step][1], points[step][2], points[step][3])
    else
        ap.push(Ev.MCE_LANDING)          -- точки кончились — садимся
    end
end

function callback(event)
    if event == Ev.ENGINES_STARTED then ap.push(Ev.MCE_TAKEOFF) end
    if event == Ev.TAKEOFF_COMPLETE then nextPoint() end
    if event == Ev.POINT_REACHED then
        Timer.callLater(1, nextPoint)    -- секунду повисеть и дальше
    end
end
```

«Дрон не долетел до высоты» Частая ошибка: задать точку и в том же обработчике через sleep(3) отправить посадку. Дрон поднимается примерно на 0.5 м/с, за 3 секунды он наберёт только полтора метра, и посадка отменит подъём. Садиться нужно по *следующему* POINT_REACHED, когда точка действительно достигнута.

Лететь медленнее или быстрее Четвёртый аргумент — время перелёта в секундах: ap.goToLocalPoint(2, 1, 1.5, 8) долетит за 8 секунд. Без него дрон летит со скоростью из параметров автопилота.

5 Таймеры, светодиоды, датчики

Таймеры

- Timer.callLater(2, function() ... end) — выполнить один раз через 2 секунды.
- Timer.new(0.5, function() ... end):start() — выполнять каждые 0.5 секунды; сохраните таймер в переменную, чтобы потом вызвать :stop().
- sleep(t) тоже есть, но он останавливает всю программу — официальная документация советует таймеры.

Светодиоды

Цвет задаётся тремя числами от 0 до 1 (красный, зелёный, синий). На плате 4 светодиода (номера 0–3), с модулем LED — 29.

```
local leds = Ledbar.new(4)
local colors = {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}} -- красный, зелёный, синий
local i = 1

function callback(event) end
```

```

Timer.new(0.5, function() -- каждые полсекунды — следующий цвет
    for n = 0, 3 do leds:set(n, table.unpack(colors[i])) end
    i = i % #colors + 1
end):start()

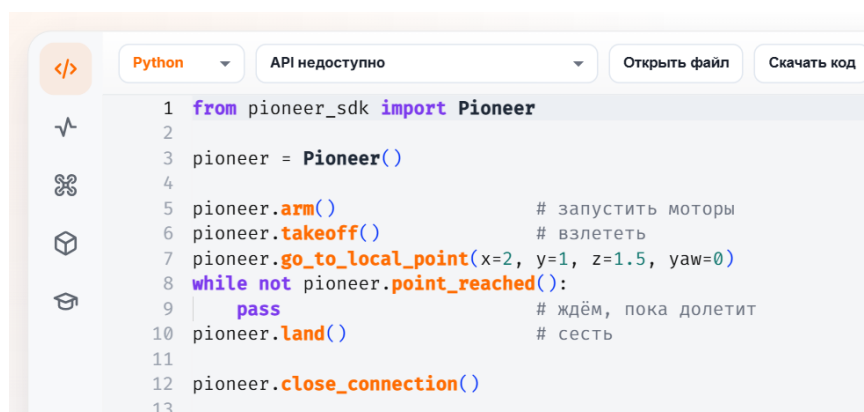
```

Датчики

local x, y, z = Sensors.lpsPosition()	положение, м
local vx, vy, vz = Sensors.lpsVelocity()	скорость, м/с
local d = Sensors.range()	дальномер: расстояние до поверхности под дроном, м
local h = Sensors.altitude()	высота по барометру, м
local yaw = Sensors.lpsYaw()	курс, радианы
local v = Sensors.battery()	напряжение батареи, В
local ch1, ..., ch8 = Sensors.rc()	каналы пульта, от −1 до 1

6 Python в браузере

Выберите в списке языка «**Python**». Это тот же `pioneer_sdk`, что и для настоящего Пионера: программа, отлаженная здесь, подходит и дрону. Python ждёт не событий, а опросом — `while not ...`



```

from pioneer_sdk import Pioneer

pioneer = Pioneer()

pioneer.arm() # запустить моторы
pioneer.takeoff() # взлететь
pioneer.go_to_local_point(x=2, y=1, z=1.5, yaw=0)
while not pioneer.point_reached():
    pass # ждём, пока долетит
pioneer.land() # сесть

pioneer.close_connection()

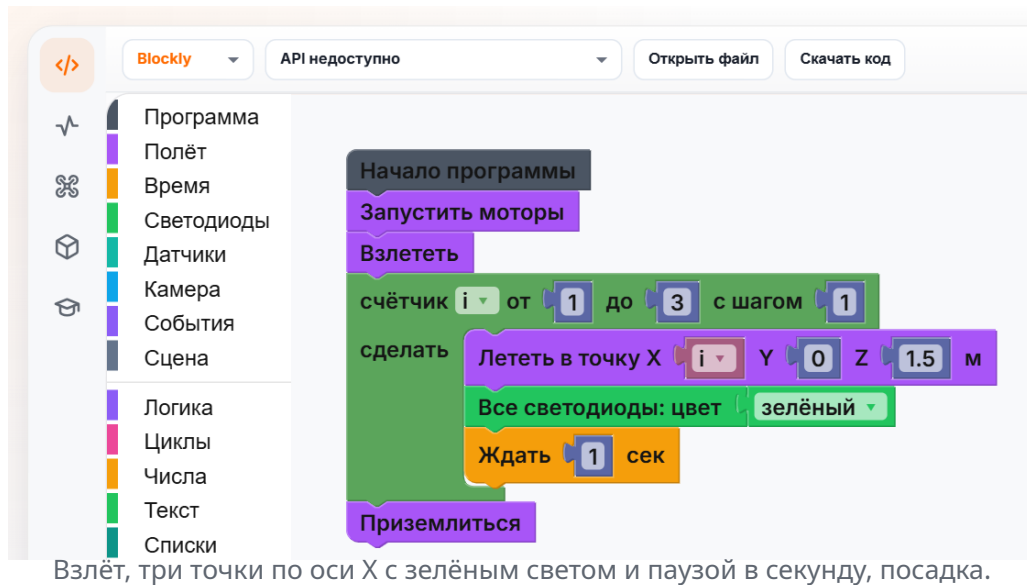
```

- `yaw` у `go_to_local_point` обязателен (радианы); `yaw=0` — курс вдоль оси X.
- Точку можно отправить сразу после `takeoff()`: автопилот полетит к ней, когда закончит взлёт.
- `point_reached()` возвращает `True` *один раз* на каждую точку.
- Цвет в Python — от 0 до 255: `pioneer.led_control(led_id=255, r=0, g=255, b=0)` (255 — все светодиоды).

- Датчики: `get_local_position_lps()`, `get_dist_sensor_data()`, `get_battery_status()`; передавайте `get_last_received=True`, чтобы повторное чтение не вернуло `None`.

7 Blockly: блоки вместо кода

Выберите в списке языка «**Blockly**» и собирайте программу из блоков под «**Начало программы**». Блоки выполняются сверху вниз: «**Взлететь**» и «**Лететь в точку**» ждут, пока дрон закончит, — события и счётчики за вас расставит сам Blockly.



- Одна и та же программа из блоков превращается и в Lua, и в Python: переключите язык — и «**Скачать код**» отдаст готовый текст, который можно дописать руками.
- Серые блоки недоступны в выбранном языке (например, ручная скорость есть только в Python, события автопилота — только в Lua). Подсказка появится, если навести курсор.
- «**Каждые N сек**» — повтор по таймеру, «**Через N сек выполнить**» — отложенное действие, «**Ждать, пока**» — пауза до выполнения условия.

8 Камеры дрона



Окна передней и нижней камер. Их кнопки — под компасом.

- Окна включаются кнопками под компасом, перетаскиваются за заголовок; двойной щелчок по заголовку возвращает окно на место. «**На весь экран**» в передней камере — вид от лица дрона на весь экран.
- Передняя камера — это кадр, который получает программа. В Lua снимок в загрузки браузера сохраняет `camera.requestMakeShot()`. В Python — как на настоящем дроне:

```
from pioneer_sdk import Camera
import cv2

camera = Camera()
frame = camera.get_cv_frame()          # кадр для OpenCV, цвета BGR
```

9 Python со своего компьютера

Программу можно запускать не в браузере, а у себя, в обычном Python с официальным `pioneer_sdk` (`pip install pioneer-sdk`), — а летать будет дрон на сайте.

1. Откройте симулятор, выберите дрон и включите у него «**Разрешить внешние команды**». Вкладку можно свернуть — полёт продолжится.
2. В программе укажите адрес сайта вместо адреса дрона:

```
from pioneer_sdk import Pioneer, Camera

pioneer = Pioneer(ip='simulator.sakoryagin.ru', mavlink_port=8001)

# Камера через интернет: один пакет от программы открывает домашний роутер для кадров.
class SimCamera(Camera):
    def connect(self):
        ok = super().connect()
        self.udp.sendto(b'1', (self.ip, self.port))
        return ok

camera = SimCamera(ip='simulator.sakoryagin.ru', port=18001)
```

10 Частые ошибки

УВЕДОМЛЕНИЯ Очистить ✕

[20:19:04] ПРЕДУПРЕЖДЕНИЕ
Проверьте сценарий перед запуском
Найдено 2 проблем в сценарии. Проверьте код перед запуском.

Команды миссии запускаются подряд без пауз. Добавьте между этапами `sleep(...)`, `Timer.callLater(...)` или `callback(event)`.

В одном шаге запускаются несколько команд миссии: `PREFLIGHT`, `TAKEOFF`. Разделите их через `sleep(...)`, `Timer.callLater(...)` или `callback(event)`.

порядок состояний дрона

`IDLE` → `PREFLIGHT` → `TAKEOFF_PROCESS` → `FLYING_HOVER` → `FLYING_MOVING` → `LANDING_PROCESS` → `IDLE`

Команды сценария должны идти в том же порядке: сначала `Ev.MCE_PREFLIGHT`, затем `Ev.MCE_TAKEOFF`, потом маршрут `ap.goToLocalPoint(...)`, и только после завершения полета `Ev.MCE_LANDING`.

Если нужно дождаться следующего этапа, продолжайте сценарий из `callback(event)`: `Ev.ENGINES_STARTED` → `Ev.TAKEOFF_COMPLETE` → `Ev.POINT_REACHED`.

Открыть 5 шагов миссии

Перед запуском симулятор проверяет программу и объясняет, что не так, в уведомлениях («колокольчик» в заголовке редактора) — вместе с исправленным примером. Слева — что он скажет на `ap.push(Ev.MCE_PREFLIGHT)` и сразу `ap.push(Ev.MCE_TAKEOFF)`.

- **Команды подряд без ожидания.** Взлёт — только после `ENGINES_STARTED`, точка — после `TAKEOFF_COMPLETE`, посадка — после `POINT_REACHED`.
- **Посадка раньше точки.** `sleep` или таймер короче перелёта — дрон сядет, не долетев.
- **Нет callback.** Выполнится только первая команда миссии.
- **Все точки на одном событии.** Без счётчика шагов вторая точка затрёт первую (раздел 4).
- **Точка «на земле».** `Z` из `Ctrl`+ЛКМ — высота поверхности, летите выше неё.
- **Python: пропущен `уав`.** На настоящем `pioneer_sdk` это ошибка.
- **Скорость и логи.** Терминал (значок `>` в заголовке редактора) показывает `print` и сообщения автопилота.

11 Шпаргалка: Lua и Python

Действие	Lua	Python
Моторы	<code>ap.push(Ev.MCE_PREFLIGHT)</code>	<code>pioneer.arm()</code>
Взлёт	<code>ap.push(Ev.MCE_TAKEOFF)</code>	<code>pioneer.takeoff()</code>
Точка	<code>ap.goToLocalPoint(x, y, z)</code>	<code>pioneer.go_to_local_point(x=, y=, z=, yaw=)</code>
Ждать точку	<code>if event == Ev.POINT_REACHED</code>	<code>while not pioneer.point_reached(): pass</code>
Курс	<code>ap.updateYaw(пад)</code>	<code>yaw= в go_to_local_point</code>
Посадка	<code>ap.push(Ev.MCE_LANDING)</code>	<code>pioneer.land()</code>
Пауза	<code>Timer.callLater(t, f)</code>	<code>time.sleep(t)</code>
Светодиоды	<code>leds:set(n, r, g, b) — 0...1</code>	<code>pioneer.led_control(led_id, r, g, b) — 0...255</code>
Позиция	<code>Sensors.lpsPosition()</code>	<code>pioneer.get_local_position_lps()</code>
Дальномер	<code>Sensors.range()</code>	<code>pioneer.get_dist_sensor_data()</code>
Батарея	<code>Sensors.battery()</code>	<code>pioneer.get_battery_status()</code>
Снимок / кадр	<code>camera.requestMakeShot()</code>	<code>Camera().get_cv_frame()</code>
Вывод	<code>print(...)</code>	<code>print(...)</code>

Где искать остальное Кнопка со значком книги в заголовке редактора открывает справочник API с описанием и примером каждой функции. Раздел «Руководство» на панели слева — уроки с заданиями и автопроверкой.